

Pixels to Predictions: Fine-Tuning SmolVLM for Multimodal Science QA

CS-GY 6953 Deep Learning Spring 2026 Final Kaggle Competition

Michael Blanchette
N16570927
mdb9910@nyu.edu

Ashley Goluoglu
N18581591
ag8529@nyu.edu

Abstract

We present our approach to the NYU Deep Learning Spring 2026 final Kaggle competition on multimodal scientific question answering. Given an image (e.g., a diagram, chart, or map) paired with a multiple-choice question, our system predicts the correct answer index. We fine-tune SmolVLM-500M-Instruct using QLoRA (4-bit quantization with Low-Rank Adaptation) under a strict 5-million trainable parameter budget. Our approach centers on a systematic ablation study spanning scoring strategy, LoRA architecture (rank, target modules, DoRA), and prompt engineering. Key findings include: (1) label masking on prompt tokens dramatically improves training signal quality, (2) higher-rank LoRA on attention-only modules outperforms lower-rank LoRA across all seven projection layers, contradicting common guidance, and (3) simpler prompt formats outperform more elaborate ones. Our best configuration, attention-only LoRA at rank 16 with a minimal prompt, achieved 0.80684 on the public leaderboard.

1 Introduction

1.1 Problem Statement

Answering science questions from visual content requires jointly understanding images and reasoning over text. Each example in this competition pairs a scientific image (a Punnett square, food web diagram, map, or chart) with a multiple-choice question and optional textual context. The model must select the correct answer index from two to five candidates. This task demands both visual interpretation and language-based reasoning, often requiring the model to extract specific information from the image and combine it with domain knowledge provided in the question context.

1.2 Motivation: Constrained Fine-Tuning

All development was constrained to Google Colab Free tier and Kaggle Free tier resources, with a

maximum of 5 million trainable parameters. These constraints shaped every decision, from model architecture to ablation strategy. With approximately 30 hours of weekly Kaggle GPU time (Tesla T4, 15GB VRAM), we needed an efficient experimentation framework that could test hypotheses quickly while producing statistically meaningful comparisons.

QLoRA, 4-bit quantization of the base model combined with low-rank adapters, offered the right tradeoff: the frozen 500M-parameter base model fits comfortably in T4 memory when quantized, while the LoRA adapters provide task-specific adaptation at a fraction of the full parameter count.

1.3 Summary of Approach

Our pipeline consists of three stages. First, we analyze the training data to understand choice distributions, subject balance, context availability, and prompt length characteristics, using these findings to guide design decisions. Second, we conduct a structured ablation study across six axes to identify the best configuration within our parameter budget: scoring strategy, DoRA, LoRA rank and target modules, and prompt engineering. Third, we perform full training with the winning configuration and generate test predictions.

Two critical fixes to the baseline training procedure proved essential: masking prompt tokens in the loss computation so the model is trained only on predicting the answer, and removing truncation from the processor calls that was silently corrupting image token counts.

2 Dataset

2.1 Description

The competition dataset is a multimodal science question-answering corpus organized into train (3,109 examples), validation (1,048), and test (1,008) splits. Each example includes an im-

age, a question, a JSON list of candidate answers (2–5 choices), and optional textual context in lecture and hint fields. Metadata columns include subject, grade, topic, category, and skill.

2.2 Data Examination

Before making any design decisions, we conducted a thorough examination of the dataset.

Choice Distribution. Approximately 50% of questions have 3 choices, 21% have 2 (binary true/false), 25% have 4, and 3.5% have 5. The weighted random baseline is 34.3%, establishing the floor against which model performance should be measured.

Subject Balance. Natural science dominates at 73%, followed by social science at 25% and language science at 2.5%. Within natural science, biology (25%) and physics (20%) are the largest topics. Crucially, the subject distribution is nearly identical across train, validation, and test splits, confirming that validation accuracy is a reliable proxy for test performance.

Context Availability. Only 14% of rows have empty lecture fields and 23% have empty hints, meaning the majority of examples include meaningful textual context. This initially suggested that including full context in prompts would be beneficial.

Prompt Length. With full context (lecture + hint), the mean prompt length is 272 tokens with a 95th percentile of 530 tokens. Approximately 7% of full-context prompts exceed 512 tokens. In contrast, hint-only prompts max out at 316 tokens with zero truncation. We addressed this by increasing `max_seq_length` to 700 rather than dropping context.

Duplicates. No duplicate IDs exist in any split. Question text is reused frequently (~63% of train rows share a question template), but these are distinct examples with different images and answers. The 204 train/validation question overlaps follow the same pattern and do not constitute data leakage.

3 Model Description

3.1 Base Model

We use SmolVLM-500M-Instruct, a vision-language model from HuggingFace with approx-

imately 303 million parameters. The model combines a vision encoder that processes images into patch embeddings with a language model that generates text conditioned on both visual and textual inputs. We load the model in 4-bit NF4 quantization with double quantization enabled, reducing memory footprint to fit within T4 constraints.

3.2 QLoRA Configuration

Low-Rank Adaptation (Schulman and Lab, 2025) is applied to the frozen quantized base model. Our final configuration uses rank 16 adapters on the four attention projection matrices (`q_proj`, `v_proj`, `k_proj`, `o_proj`), with $\alpha = 16$ (scaling factor 1.0) and dropout of 0.05. This yields 4,161,536 trainable parameters, consuming 83% of the 5M budget.

Image splitting in the processor is disabled (`do_image_splitting = False`), reducing each image to a single patch rather than up to 17 tiles. This was essential for memory management, with splitting enabled, each sample could consume ~85MB, causing out-of-memory errors on the T4.

3.3 Critical Fix: Label Masking

The most impactful change to the training procedure was implementing proper label masking. The baseline approach sets `labels = input_ids.clone()`, computing the cross-entropy loss over the *entire* sequence: every context token, question token, and choice token. This massively dilutes the learning signal, as the model spends most of its gradient updates learning to reconstruct the prompt rather than predicting the correct answer.

Our fix masks all prompt tokens with `-100` (PyTorch’s ignore index), so the loss is computed only on the answer token(s). The implementation works backward from the end of the tokenized sequence: it identifies the last non-padding token, tokenizes the answer portion separately to determine how many tokens to unmask, and sets only those positions to their true token IDs in the labels tensor. The model still attends to the full prompt through its attention mechanism; this change affects only what the optimizer trains on.

3.4 Critical Fix: Processor Truncation

The processor calls in the scoring utilities originally included `max_length` and `truncation=True` arguments. SmolVLM’s processor expands `<image>`

placeholder tokens internally into many tokens representing image patches. Truncating before this expansion completes caused a mismatch between the number of image tokens in the text and in the `input_ids` tensor, crashing with a `ValueError`. Removing these arguments and letting the processor handle sizing natively resolved the issue.

4 Experimentation

4.1 Ablation Framework

We built a systematic ablation framework that tests one variable at a time, with each run producing comparable results under identical conditions. Each ablation run trains for 300 steps (approximately 38% of one epoch) with evaluation every 150 steps on a fixed subset of 200 validation examples. Results are persisted to JSON between sessions to accommodate Kaggle’s session limits.

After initial experiments with 200-step ablations, we found that these evaluation points were insufficient to distinguish close contenders and increased to 300 steps with evaluation at steps 150 and 300. This provided enough signal to better identify winners while keeping each run under 12 minutes.

4.2 Axis 1: Scoring Strategy

We compared two inference strategies: **generate** (run `model.generate()`, parse the output letter) and **log-likelihood** (for each choice, compute the log-probability the model assigns to that choice’s letter token, pick the `argmax`).

Our log-likelihood implementation extracts the logit at the position predicting the answer token (position -2 in the sequence, since causal LM logits at position t predict token $t + 1$), applies log-softmax, and scores the candidate token ID. This is more precise than using `outputs.loss`, which averages over all tokens.

Strategy	Step 150	Step 300	Time
Generate	0.590	0.610	11.6 min
Log-likelihood	0.620	0.550	11.9 min

Table 1: Axis 1: Scoring strategy. Log-likelihood peaked early but collapsed by step 300, suggesting instability. Generate was selected for its stable trajectory.

Log-likelihood peaked at 62% at step 150 but dropped to 55% by step 300, indicating instability as training progressed. Generate showed steady improvement. We selected generate for its reliability.

4.3 Axis 2: DoRA

DoRA (Weight-Decomposed Low-Rank Adaptation) decomposes weight updates into magnitude and direction components. Since DoRA with all seven modules at rank 8 exceeded the 5M budget (5.09M), we compared no-DoRA at rank 8 against DoRA at rank 6.

Config	Params	Step 150	Step 300	Time
No DoRA, r=8	4.78M	0.570	0.610	11.5 min
DoRA, r=6	3.89M	0.600	0.600	23.1 min

Table 2: Axis 2: DoRA. Standard LoRA outperformed DoRA while taking half the training time.

DoRA doubled training time (23 vs. 12 minutes) due to the magnitude/direction decomposition overhead and did not improve accuracy. This pattern held across all subsequent comparisons: DoRA consistently underperformed standard LoRA on this task.

4.4 Axis 3: Architecture: Rank and Target Modules

A key question was whether to target all seven projection layers (attention + MLP) at lower rank or fewer layers at higher rank. Course guidance suggested expanding to MLP layers (gate_proj, up_proj, down_proj) at lower rank. Our results contradicted this.

Config	Params	Step 300	Time
attn4, r=16	4.16M	0.680	10.1 min
attn4, r=16, DoRA	4.27M	0.650	14.6 min
all7, r=8	4.78M	0.610	10.6 min
all7, r=6, DoRA	3.89M	0.670	20.5 min
all7, r=4, DoRA	2.70M	0.650	20.9 min

Table 3: Axis 3: LoRA architecture. Attention-only at rank 16 outperformed all seven modules at lower ranks, including with DoRA.

Attention-only at rank 16 (68%) consistently outperformed all-seven-modules at rank 8 (61%) across every prompt variation tested. This suggests that for this task, the model benefits more from higher-capacity attention adaptation than from broader but thinner coverage of both attention and MLP layers.

4.5 Axis 4: Prompt Engineering

We tested four prompt variations across both architectures to assess the impact of metadata (subject

and grade fields) and answer format (“Answer:” vs. “The correct answer is:”).

Prompt Config	attn4 r=16	all7 r=8
Bare (no meta, “Answer:”)	0.690	0.600
+ Metadata only	0.680	0.610
+ Format only	0.600	0.610
+ Metadata + Format	0.680	0.610

Table 4: Axis 4: Prompt engineering across both architectures. The bare prompt consistently performs best on attn4 r=16.

The simplest prompt (no metadata, “Answer:” format) achieved the highest accuracy (69%). “The correct answer is:” format actually hurt performance, possibly because the longer format consumes tokens without adding discriminative signal for a model that has already seen the question and choices. Metadata showed a neutral to slightly negative effect, suggesting the model does not benefit from explicit subject/grade labels at this scale.

4.6 Ablation Summary

Table 5 presents all 15 ablation runs sorted by final validation accuracy. Figure 1 shows validation accuracy trajectories across all 15 ablation runs.

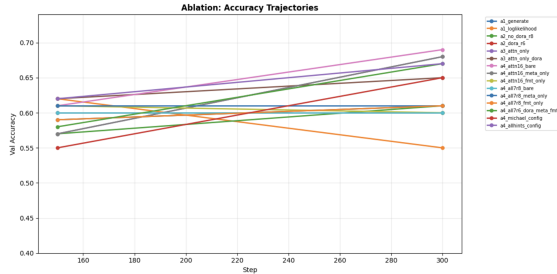


Figure 1: Validation accuracy trajectories across all ablation runs. The attn4 r=16 configurations (pink, gray, purple) consistently separate from the all7 r=8 cluster (blue, cyan, orange) after step 200, confirming that the architecture difference is robust rather than a single-point measurement artifact.

5 Results

5.1 Final Configuration

Based on the ablation study, our final training configuration was:

- **Scoring:** generate
- **LoRA:** rank 16, $\alpha = 16$, dropout 0.05, attention-only (q/v/k/o_proj)

- **DoRA:** disabled
- **Prompt:** no metadata, “Answer:” format, full context (lecture + hint)
- **Training:** 3 epochs + 3 additional epochs at half learning rate (1×10^{-4}), batch size 4 with 4-step gradient accumulation (effective batch 16), cosine schedule with 10% warmup
- **Sequence length:** 700 tokens
- **Trainable parameters:** 4,161,536 (83% of 5M budget)

5.2 Leaderboard Performance

Our system achieved a score of **0.80684** on the public leaderboard, representing a classification accuracy of 80.7% on the hidden test set. Starting from a random baseline of 34.3% (weighted by choice count) and a first-ablation accuracy of 61%, the combination of label masking, architecture optimization, and extended training yielded substantial gains.

5.3 Training Progression

The initial 3-epoch training run took approximately 90 minutes on the Kaggle T4. Accuracy climbed steadily through all three epochs, reaching 79% on the full validation set. We then resumed training for 3 additional epochs at half the learning rate (1×10^{-4}) to allow finer convergence, which pushed accuracy to the final 80.7%. Checkpoints were saved after each epoch to enable resumption across Kaggle sessions.

6 Discussion

6.1 Key Findings

Label masking is essential. The single largest improvement came from masking prompt tokens in the loss. Before this fix, accuracy plateaued around 46% in ablation runs. After, the same configuration reached 61%, rising further with architecture and prompt optimizations. The insight is simple: training the model to reconstruct the entire prompt wastes gradient signal on a task orthogonal to answer prediction.

Higher rank on fewer modules beats broader coverage at lower rank. This contradicts common LoRA guidance and published findings suggesting MLP adaptation is important. For our task,

Run	Scoring	Rank	Modules	DoRA	Meta	Format	Params	Acc	Time
a4_attn16_bare	generate	16	attn4	No	No	Answer:	4.16M	0.690	9.7
a3_attn_only	generate	16	attn4	No	Yes	Correct ans.	4.16M	0.680	10.1
a4_attn16_meta	generate	16	attn4	No	Yes	Answer:	4.16M	0.680	9.3
a4_all7r6_dora	generate	6	all7	Yes	Yes	Correct ans.	3.89M	0.670	20.5
a4_allhints	generate	4	all7	Yes	Yes	Correct ans.	2.70M	0.670	20.7
a3_attn_dora	generate	16	attn4	Yes	Yes	Correct ans.	4.27M	0.650	14.6
a1_generate	generate	8	all7	No	Yes	Correct ans.	4.78M	0.610	11.6
a4_all7r8_meta	generate	8	all7	No	Yes	Answer:	4.78M	0.610	10.7
a4_all7r8_fmt	generate	8	all7	No	No	Correct ans.	4.78M	0.610	10.6
a4_attn16_fmt	generate	16	attn4	No	No	Correct ans.	4.16M	0.600	9.3
a4_all7r8_bare	generate	8	all7	No	No	Answer:	4.78M	0.600	10.6
a2_dora_r6	generate	6	all7	Yes	Yes	Correct ans.	3.89M	0.600	23.1
a1_loglikelihood	loglik.	8	all7	No	Yes	Correct ans.	4.78M	0.550	11.9

Table 5: Full ablation results sorted by validation accuracy. Modules: attn4 = q/v/k/o_proj; all7 = attn4 + gate/up/down_proj. “Correct ans.” = “The correct answer is:”. All runs: 300 steps, eval subset of 200 validation examples.

the model needed higher-capacity attention adaptation (rank 16 on 4 modules) rather than thinner adaptation across all 7 projection layers (rank 8). One hypothesis is that for visual question answering, the attention mechanism, which determines how the model relates image patches to question tokens, is the primary bottleneck, while the MLP layers’ feed-forward transformations are less task-specific.

Simpler prompts outperform elaborate ones. Adding subject/grade metadata and using a longer answer format (“The correct answer is:”) did not help and in some cases hurt. The model appears to extract sufficient signal from the question, choices, and image without explicit metadata cues. The longer answer format may consume tokens without adding discriminative information.

DoRA adds cost without benefit. Despite documented gains on VLM benchmarks, DoRA consistently underperformed standard LoRA on our task while doubling training time. This may be task-specific: the magnitude/direction decomposition may not benefit a multiple-choice classification task as much as open-ended generation tasks.

6.2 Limitations

Ablation noise. With 200 evaluation examples and 300 training steps, differences of 1–2% (2–4 examples) are within noise. Some of our per-axis decisions (particularly scoring strategy, where generate and log-likelihood tied at 61%) may not be statistically significant.

Image resolution. We let the processor handle image resizing (384×384 natively), but the course

hints suggested larger images might preserve important details in diagrams. We did not systematically test this due to memory constraints.

Captions not tested. Pre-generating image captions, using the model itself to describe each image, then feeding descriptions as text context, was a suggested technique we implemented but did not have time to ablate. Initial inspection of the generated captions showed that the model was largely generating nonsense, and we concluded it would likely harm rather than help.

Potential ceiling. The gap between our 80.7% and the class high of 99.2% suggests substantial room for improvement, potentially from caption augmentation, larger image sizes, or more sophisticated training strategies.

6.3 Future Work

Given more time and resources, we would: (1) test the impact of pre-generated image captions as an ablation axis, (2) test larger image resolutions (512×512) with gradient checkpointing to manage memory, (3) run longer ablations (500+ steps) to reduce evaluation noise, (4) implement choice-order shuffling during training to reduce positional bias, and (5) explore ensemble methods combining generate and log-likelihood scoring at test time.

7 Conclusion

We developed a multimodal science QA system achieving 80.7% accuracy on the competition test set, fine-tuning SmolVLM-500M-Instruct with QLoRA under a 5M trainable parameter budget. Our systematic ablation study across six axes iden-

tified an optimal configuration of attention-only LoRA at rank 16 with a minimal prompt format. The most impactful finding was that proper label masking (training only on answer tokens rather than the full sequence) improved accuracy by approximately 15 percentage points. We also found that common fine-tuning heuristics (broader LoRA targets, DoRA, metadata-enriched prompts) did not transfer to our specific task, underscoring the importance of empirical validation over received wisdom.

7.1 AI Tooling Disclosure

We used Claude as a coding and debugging assistant throughout development, primarily for debugging processor and tokenizer interactions. All experimental design decisions, including which ablation axes to test and how to interpret results, were made by the team.

7.2 Links

Resource	Link
GitHub Repository	https://github.com/agoluoglu/Pixels-to-Predictions
Kaggle Competition	https://www.kaggle.com/competitions/pixels-to-predictions/overview

References

John Schulman and Thinking Machines Lab. 2025. [Lora without regret](https://thinkingmachines.ai/blog/lora/). *Thinking Machines Lab: Connectionism*. <https://thinkingmachines.ai/blog/lora/>.